1. Quick Start: CloneMC Folder Includes

Purpose of this guide

- This PDF documents the uploaded CloneMC_SingleplayerAndMultiplayer.zip file from GitHub mainly. The zip contains a V74 Open Watcom/Win32/OpenGL build in folder CloneMC_OpenWatcom_V74_JavaTerrain_Zlib_Performance.
- The project is intentionally a unity build: compile project2finalalpharecreation.c only. That root file #includes the split src/ folders in a fixed order. The src files are editable sections, not separate object files.
- This guide explains where the main systems live, what the important fields control, how to edit textures/models/particles/menus/multiplayer, and how to compile the folder with Open Watcom.

Root files to know

- project2finalalpharecreation.c - master source driver. It includes core, network, save, entity, item, world, render, lighting, UI, and scheduled tick sections.
- project1finalalpha.c - compatibility wrapper/name used by some older scripts, but the current Makefile builds project2finalalpharecreation.c.
- Makefile.v74.wat - primary Open Watcom makefile. Use wmake -f Makefile.v74.wat from the project root.
- clonemc_v74_nt_win.lnk - linker response file. Libraries are opengl32, glu32, gdi32, user32, winmm, and ws2_32.
- assets/ - terrain, items, GUI, mob textures, particles, environment, title, armor, art, sounds/music references, and texture pack support.
- DLLS/zlib.dll - uploaded zlib DLL for multiplayer terrain decompression. Modern Windows likely works; Windows 98 may need an older Win9x-compatible zlib DLL as mentioned in github.

Fast Ctrl+F searches

- Use Ctrl+F in your editor before editing. Search function names instead of scrolling: RenderWorld, GameRender, BuildTerrainChunkMesh, RenderMobs, JavaModel_RenderCow, SpawnParticleV24, InitSurvival, GuiV9_DrawMultiplayer, NetworkV60_Tick, GetTerrainTileUV.

2. Source Folder Map and Core Data Fields

src folder map

- src/00_core/clonemc_core_defs.c - constants, block IDs, item IDs, structs, globals, forward declarations, texture tile coordinates, world sizes, render knobs, player state, GUI state.
- src/10_save/clonemc_save_tile_region.c - save paths, world/player/entity/tile save/load, simple NBT-like writers/readers, region chunk persistence, tile entity serialization.
- src/20_entity/ - mob AI, movement, player physics, mob rendering, first-person held item/arm, block breaking/placing raycasts, dropped item rendering.
- src/30_item/ - inventory, crafting, furnace/container UI, item use, combat, drops, redstone, weather, special entities such as boats/minecarts/projectiles.
- src/40_world/ - world generation, finite/infinite streaming, biome helpers, caves/ores/trees/decorations, particles, dropped item update loop, FPS/frame pacing.
- src/50_lighting/ - sky light, block light, propagation, local light updates, brightness/AO helpers used by renderers.
- src/60_ui/ - Win32 window, OpenGL context, textures, fonts, menus, GUI screens, chat input, world select/create, options, save slot entry points.
- src/70_render/ - block render types, chunk display lists, frustum queues, transparent pass, selection outlines, special entity rendering, first-person item rendering.
- src/NETWORK/ - WinSock2 Beta/Classic networking, packet parsing/writing, zlib terrain decompression, username/protocol selection, chat safety.

Important structs and fields

- InventorySlot { item, count, damage } - used by hotbar, main inventory, armor, crafting, containers, and save/load. Add new item durability through damage.
- Mob - active/type/health/position/velocity/yaw, AI timers, path nodes, hurt/death timers, animation walk values, entity state flags, and persistent flag. Mob rendering and AI both read this struct.
- Particle - position, velocity, life/maxLife, block, type, variant, size, gravity, age, noClip. Particle drawing chooses texture and color from type/block.
- DroppedItem - item/count/damage, position/velocity, age, spin, hoverStart, pickupDelay. Rendered as a spinning card/cube depending on item type.
- WorldSaveInfo - slot metadata: name, seed text, seed, world size, player/global position, spawn, keepInventory, worldTime.
- GuiV9TextField - rect, text, maxLen, focused, cursor, selection, viewOffset, enabled. Multiplayer address/name and chat fields use this style.

Important global arrays

- world[WORLD_X][WORLD_Y][WORLD_Z] - block IDs for the active 128x128x128 window. Do not confuse this RAM window with the finite 862/custom world size.
- g_blockMeta - block metadata, used by torches, redstone, doors, rails, water/lava, and other directional/stateful blocks.
- skyLight and blockLight - lighting arrays. Rebuild with ComputeLegacyLighting or RecomputeLegacyLightingLocal after block batch changes.
- columnTop/worldHeightMap/biomeMap - generated terrain summaries used by spawn, culling, weather, mobs, and chunk rendering.
- mobs, droppedItems, particles, tileEntities, g_specialEntitiesV6 - fixed-size arrays for Win98-friendly memory and no dynamic allocation spikes.

.3. OpenGL Rendering Pipeline and Performance Knobs

OpenGL setup and main loop

- InitOpenGL in src/60_ui/clonemc_platform_ui_resources.c creates the Win32 OpenGL context, enables depth/texture/culling basics, loads wglSwapIntervalEXT for VSync, and loads game textures.
- MainLoop pumps Win32 messages, clamps dt to 0.05 seconds, updates music/resources/network, runs GameUpdate while in STATE_GAME, then calls GameRender and FramePacerV59_End.
- Frame pacing is in src/40_world/clonemc_world_generation_streaming.c: SetFrameLimitExactV59, ApplyVSyncV59, FramePacerV59_Begin, FramePacerV59_End. Default is 30 FPS, with 60 FPS and VSync options.

GameRender order

3. GameRender in src/60_ui chooses menu screens by g_state. For game/death/pause states it sets projection, calls SetupCamera, renders sky, fog, world chunks, transparent/special blocks, entities, mobs, clouds, particles, first-person item, HUD, chat, pause/death overlays, then SwapBuffers.
4. Important render call order: RenderWorld -> RendererV8_RenderSpecialWorldBlocks -> RendererV8_RenderTransparentBlocks -> RenderDroppedItems -> RendererV8_RenderSpecialEntities -> RenderMobs -> RenderParticles -> RendererV8_RenderFirstPersonItem -> HUD/chat.
5. The order matters: solid chunks first, transparent blocks after, entities after terrain, particles near the end, and 2D GUI after world rendering.

Chunk rendering

6. BuildTerrainChunkMesh in src/40_world builds OpenGL display lists for solid chunk geometry. It uses DrawBlock/DrawFace/RenderBlockAtV19 and marks terrainChunkDirty false after rebuilding.
7. src/70_render/clonemc_render_chunks_special_entities.c builds sorted chunk queues with RendererV33_BuildChunkQueues. It culls by distance/frustum, uses display lists, and limits rebuilds per frame.

8. Performance knobs live in src/00_core: g_chunkMeshBuildBudget, g_renderV47MaxBuildMs, g_renderV47TransBuildBudget, g_renderV41FullMeshRadiusChunks, g_particleCullDistanceBlocks, g_mobFullModelDistanceBlocks.
9. If there are lag spikes, reduce g_chunkMeshBuildBudget or g_renderV47MaxBuildMs. If chunks appear too slowly, increase them slightly. Avoid rebuilding all chunks every block change; use InvalidateTerrainChunkMeshAt for local edits.

How to debug render problems

10. Ctrl+F RenderWorld to inspect the per-frame world path. Ctrl+F BuildTerrainChunkMesh for chunk geometry. Ctrl+F RendererV33_GetMeshBuildBudget for stutter. Ctrl+F terrainChunkDirty to see what triggers rebuilds.
11. If something renders but with the wrong texture, start with BlockV19_GetTextureTileAt and GetTerrainTileUV. If it does not render at all, start with RenderBlockAtV19 or the block registry render type.

Textures, UV Mapping, Blocks, Leaves, and Water

Terrain atlas and tile coordinates

- Terrain tile coordinates are centralized in src/00_core/clonemc_core_defs.c under TILE_*_COL and TILE_*_ROW. The project uses a 512x256 terrain atlas, so columns/rows are atlas tile positions, not pixels.
- GetTerrainTileUV in src/60_ui converts tile col/row into normalized OpenGL UV coordinates. It is called block faces, item cards, particles, and fluid rendering.
- BlockV19_GetTextureTileAt and GetBlockTextureTile select which tile a block face uses. Edit those functions when a block needs different top/side/bottom textures, metadata-based faces, or animated/special tiles.
- assets/terrain.png and assets/terrain.tga are the main terrain sheets. V74 also includes assets/terrain_v58_canonical.* as a reference/fallback.

Block render types

- src/70_render/clonemc_render_block_types.c contains block geometry renderers: RenderRailBlockV43, RenderRedstoneWireV43, RenderPressurePlateV43, RenderButtonV43, RenderLeverV43, RenderTrapdoorV43, RenderChestBlockV43, RenderStairsBlockV43, RenderPistonBlockV43, RenderBedBlockV43, RenderFireBlockV43, RenderSlabOrLayerBlockV57, DrawTorchBlock, DrawBlock, DrawFace.
- Block registry setup is in src/60_ui around BlockRegistryV19_Init and later override passes. Registry fields control material, collision, hardness, harvest tool, render type, render layer, replaceability, support, and drops.
- To add a block: define BLOCK_* and ITEM_* in core, define tile coordinates, add registry entry, add texture selection, add render type if not a cube, update collision/support/drop logic, and add crafting/placement if needed.

Leaves and biome tint

- Leaves use BLOCK_LEAVES and texture globals. Their atlas tile defaults are TILE_LEAVES_COL/ROW in core. Rendering tint is handled through DrawBiomeTintOverlayForBlock and color logic around SetBlockColorFallback / block render calls.
- Leaf problems are usually caused by one of three places: wrong atlas coordinates, missing alpha/transparent render layer, or foliage color not applied. Search BLOCK_LEAVES, TILE_LEAVES, RendererV8_IsTranslucentBlock, and DrawBiomeTintOverlayForBlock.

Water texture and fluid rendering

- Water block ID is BLOCK_WATER. Still/flow atlas tiles are TILE_WATER_STILL_COL/ROW and TILE_WATER_FLOW_COL/ROW in core. Water render height and shape are in FluidV42_GetRenderedHeight and RenderFluidBlockV42 in src/30_item/clonemc_inventory_crafting_ui_tail.c.
- Water also uses ResourceV10 animation state: g_textureFxWaterFrameV54 and beta water resources such as assets/beta/water.png, assets/misc/water.png, and assets/beta/watercolor.png.
- If water looks wrong, Ctrl+F TILE_WATER, RenderFluidBlockV42, texBetaWater, ResourceV10_UpdateAnimations, and BlockV57_GetRenderLayer.

Mob Rendering, Mob Textures, and Mob Models

Where mob rendering lives

- Main rendering file: src/20_entity/clonemc_entity_mob_render_player.c. Important functions include RenderMobs, RenderMobModelTex, RenderMobModelJavaV53, JavaModel_RenderExactMob, JavaModel_RenderCow, JavaModel_RenderPig, JavaModel_RenderSheepBase, JavaModel_RenderChicken, JavaModel_RenderSpider, JavaModel_RenderWolf, JavaModel_RenderCreeperV53, JavaModel_RenderSlimeV53, JavaModel_RenderSquidV53.
- Mob interpolation is in src/20_entity/clonemc_entity_mob_ai_core.c: MobInterp_SyncV32, MobInterp_BeginStepV32, MobInterp_LerpV32, MobInterp_LerpAngleV32, MobInterp_BuildRenderMobV32. This keeps mob motion smoother than fixed AI ticks alone.
- Mob shadows are drawn by DrawMobShadow. Billboards/fallbacks use DrawMobBillboard. Full model distance is controlled by g_mobFullModelDistanceBlocks in core.

Texture globals and asset paths

- Mob GL textures are globals in src/00_core: texMobChicken, texMobCow, texMobSheep, texMobSheepFur, texMobWolf, texMobWolfAngry, texMobWolfTame, texMobSquid, texMobZombie, texMobSkeleton, texMobCreeper, texMobSpider, texMobSpiderEyes, texMobSlime, texMobPig, texMobPlayer.
- Asset folders include both assets/mob/ and assets/mobs/. Examples: assets/mob/cow.png, assets/mob/sheep_fur.png, assets/mob/spider_eyes.png, assets/mob/char.png and duplicate/fallback assets/mobs/*.png.
- Texture loading and aliases are in src/60_ui: ResourceV10_ResolveTextureFile, ResourceV10_LoadTGATexture, ResourceV10_ReloadTextures, LoadGameTextures. If a texture does not load, check file path, PNG/TGA conversion, and fallback aliases

How models use UVs

- Java-style model parts are assembled with JavaPTV_SetTexturePosition, JavaQuad_SetTextureRect, JavaModel_RenderPart, and per-mob render functions. This matches the Java model idea: cuboids with texture offsets inside mob skin files.
- If a model has scrambled texture faces, search JavaModel_RenderPart and the model-specific function. The UV rectangles are normally copied from Java Model* classes as texture offsets and cuboid sizes.
- Spider eyes use a separate texture pass through texMobSpiderEyes. Sheep render base wool/body and a fur overlay when not sheared.

Adding or editing a mob

- 1. Add or confirm a MOB_* type constant in core. 2. Add texture globals and load paths in LoadGameTextures. 3. Add health in MobMaxHealthJavaV4. 4. Add AI behavior in mob_ai_core or reuse passive/hostile logic. 5. Add model render branch in JavaModel_RenderExactMob or RenderMobModelTex. 6. Add loot in DropMobLoot. 7. Add save/load compatibility through SaveMobToBytes/Load. 8. For multiplayer, map server entity IDs in src/NETWORK.

Mob AI, Particles, Drops, and Survival Mode

Mob AI and movement

- src/20_entity/clonemc_entity_mob_ai_core.c controls walking, pathing, fluid states, hazards, attacks, pushing, jumping, drowning/burning, and target selection. Key functions: MobAI_SelectTargetKindV24, MobAI_BuildPathV24, MobAI_GetCachedPathSteerV24, MobAI_AttackPlayerV24, MobAI_ResolveAxisSweepV24.
- UpdateMobs in src/40_world drives mob logic. GameUpdate runs mob logic at a fixed 20 Hz accumulator using MOB_LOGIC_STEP_SECONDS and MOB_LOGIC_MAX_STEPS_PER_FRAME, then render interpolation fills visual motion between ticks.
- Mob struct fields to watch: health, hurtTime, deathTime, deathDropsDone, animWalk, renderYawOffset, pathLength/pathIndex, targetKind, onGround/inWater/inLava, attackCooldown, persistent.

Particles

- Particle definitions are in core. Runtime functions are in src/40_world: InitParticles, GetParticleColorForBlock, SpawnParticleV24, SpawnSmokeParticlesV24, SpawnFlameParticlesV24, SpawnSplashParticlesV24, SpawnExplosionParticlesV24, SpawnBlockBreakParticles, SpawnMobEffectParticles, UpdateParticles, ParticleV54_GetParticleTile, ParticleV54_GetColorAndAlpha, RenderParticles.
- Particle texture source is usually assets/beta/particles.* or assets/gui/particles.* loaded as texBetaParticles. Block break particles often sample terrain atlas color by block ID.
- To add a particle: define PARTICLE_* in core, spawn it with SpawnParticleV24, add tile/color behavior in ParticleV54_GetParticleTile and ParticleV54_GetColorAndAlpha, then call it from block/item/mob code.

Survival inventory and HUD

- Inventory starts in src/30_item/clonemc_inventory_crafting_ui_tail.c. InitSurvival initializes hotbar, inventory, armor/crafting arrays. DrawSurvivalUI draws hotbar/health and inventory overlays. InventoryMouseClick/RightClick handle stack movement.
- Hotbar/inventory arrays are in core: hotbar[HOTBAR_SLOTS], inventory[INVENTORY_SLOTS], armorSlots, craftGrid/craftResult. InventorySlot has item/count/damage.
- Health/death are tied into DrawDeathScreen in UI, TakeDamage/playerHeartsLife in core, and PlayerV66_HandleDeathInventory for keep-inventory behavior.

Block breaking, drops, and combat

- Block mining and combat are in src/30_item/clonemc_item_combat_special_entities.c. Key functions: ItemCombatV6_BlockHardness, ItemCombatV6_CanHarvestBlock, BeginHeldBlockMiningV11, UpdateHeldBlockMiningV11, FinishMinedBlockV11, TryContinuousAttackMobV28.
- Block drops: GetBlockDropItemAt in inventory tail, BlockV50_GetDropItemAt in UI/block registry, BlockV50_DropBlockAsItem, and DropItemAt/SpawnDroppedItem functions in world/player code. Mob drops: DropMobLoot in containers/redstone and KillMobRenderable.
- If a block or mob gives nothing, Ctrl+F its BLOCK_* or MOB_* type in GetBlockDropItemAt, BlockV50_GetDropItemAt, DropMobLoot, and KillMobRenderable.

World Generation, Infinite/862 Worlds, Lighting, and Saves

How the active world window works

- The RAM world is a 128x128x128 active window defined by WORLD_X/Y/Z. Infinite and finite/custom worlds are not fully held in memory; they stream deterministic/generated chunks into the active window using worldOriginBlockX/worldOriginBlockZ.
- GenerateWorld and GenerateWorldWindow live near the top of src/40_world/clonemc_world_generation_streaming.c. UpdateInfiniteWorldStreaming moves the active window when the player approaches the edge and saves/loads generated regions.
- Finite world size is stored in g_worldSizeBlocks. WORLD_SIZE_INFINITE is 0, and finite/default size is 862 through FINITE_WORLD_DEFAULT_SIZE and newWorldSizeText.

Terrain generation functions

- Core terrain helpers in src/40_world: WorldPerlin2D, WorldPerlin3D, WorldGenV3_ComputeClimate, WorldGenV3_BiomeFromClimate, GetBetaBiomeAt/GetBiomeAtGlobal, BiomeTopBlock, BiomeFillerBlock, BetaMountainMask, BetaDensity3D, BetaTerrainHeight.
- Content passes include AddBetaCaveWorms, IsBetaCave, AddBetaTree, AddBigBetaTree, AddBetaCactus, WorldGenV3_AddOreVeinJava, WorldGenV20_AddJavaOrePass, WorldGenV20_AddDecorationPass, AddSurfaceTexturePass, and cleanup/surface passes.
- Grass/sand problems usually happen in biome top/filler replacement: search BiomeTopBlock, BiomeFillerBlock, WorldGenV3_TopBlockForBiome, WorldGenV3_FillerBlockForBiome, WorldGenV58_SetSurfaceColumn.

Lighting

- src/50_lighting/clonemc_lighting_brightness_render.c owns light arrays. ClearLightArrays resets them. GetBlockLightValueSourceV48 gives emissive blocks. ComputeLegacyLighting rebuilds global light. RecomputeLegacyLightingLocal handles localized changes.
- Renderers sample GetLightLevel, GetMixedLightLevelV48, ClampLightFloat, VertexAOFromBlocks. If chunks are black, stale, or overly bright, inspect lighting rebuild order after generation/chunk apply/block edits.
- After many terrain changes, the safe order is: generate or modify blocks -> RebuildColumnTops -> ComputeLegacyLighting or local light update -> InvalidateTerrainChunkMeshAt or InvalidateAllTerrainChunkMeshes.

Save/load

- World slot metadata, player, entities, drops, tile entities, and regions are saved through src/10_save/clonemc_save_tile_region.c plus wrapper functions in src/60_ui. Main entry points: SaveCurrentWorld, SaveHandler_SaveCurrentWorldV2, SaveHandler_LoadWorldInfoV2, SaveHandler_LoadRegionWindowV2, SaveHandler_SaveRegionWindowV2.
- Save paths include saves/WorldN.dat, WorldN_blocks.rle, WorldN_inventory.dat, WorldN_mobs.dat, WorldN_drops.dat, and region-style files under saves/worldN_region_lite.mcr / r.*.mcr depending on version.

Menus, GUI, Input, Chat, and Texture Packs

Game states and menus

- GameRender switches on g_state. Common states include STATE_MENU, STATE_WORLD_SELECT, STATE_CREATE_WORLD, STATE_OPTIONS, STATE_VIDEO_SETTINGS, STATE_CONTROLS, STATE_MULTIPLAYER, STATE_CONNECTING, STATE_DOWNLOAD_TERRAIN, STATE_TEXTURE_PACKS, STATE_PAUSE, STATE_DEATH.
- Main menu buttons are in src/60_ui around LayoutBetaMenus and DrawMenu. Click handling lives in WindowProc WM_LBUTTONDOWN around the state-specific button checks.
- World select/create functions: EnterWorldSelect, DrawWorldSelect, EnterCreateWorld, DrawCreateWorld, HandleCreateWorldChar, CreateWorldFromMenu, StartNewWorldInSlot, StartWorldSlot. World size and keep inventory controls are in that create-world area.
- Video/options functions: EnterOptions, DrawOptions, GuiV7_EnterVideoSettings, GuiV7_DrawVideoSettings. Frame limit and VSync UI route to SetFrameLimitExactV59 and ToggleVSyncV59.

Text fields and chat

- GuiV9TextField handles editable text. Functions: GuiV9_TextFieldInit, GuiV9_TextFieldDraw, GuiV9_TextFieldChar, GuiV9_TextFieldKey, GuiV9_TextFieldMouse. These are used by multiplayer address/username and other GUI fields.
- Chat overlay: GuiV9_AddChatLine, GuiV9_DrawChatOverlay, GuiV9_HandleChar, GuiV9_HandleKeyDown. T opens chat; slash commands are sent to the server when connected instead of being executed locally.
- Visible text cleanup uses GuiV65_SanitizeVisibleText and network-side NetV61_SanitizeDisplayText to remove odd color-code bytes and bad characters from server messages.

Texture packs and resources

- ResourceV10_* functions in src/60_ui load default assets and optional texture packs. ResourceV10_ResolveTextureFile searches active pack/fallback paths. ResourceV10_ReloadTextures reloads when pack changes.
- Texture pack menu: GuiV9_DrawTexturePacks, GuiV9_TexturePackSelect, GuiV9_DrawConflictWarning. Packs live under assets/texturepacks and may use packs.txt plus cached folders/ZIPs.
- Fonts: CreateFonts creates Win32 Arial display-list fonts; DrawBetaFontText2D/DrawBlockyText2D draw GUI text. HUD status text is DrawBetaStatus2D in src/40_world, showing version, XYZ, and FPS.

Editing GUI safely

- To add a button: add a RECT global in core if it must persist, position it in LayoutBetaMenus or the screen draw/setup function, draw it with DrawButton2D/GuiV9 helpers, then handle clicks in WindowProc for that state.
- If typing does not work, Ctrl+F WM_CHAR, GuiV9_HandleChar, GuiV7_HandleChar, HandleCreateWorldChar. If clicks do not work, Ctrl+F WM_LBUTTONDOWN and the state name.

Multiplayer, Networking, Server Chunks, and zlib

Network file overview

- src/NETWORK/clonemc_network_beta173.c is the multiplayer layer. It uses WinSock2, fixed buffers, non-blocking connect/read, packet readers/writers, Beta and Classic protocol modes, zlib decompression, and GUI status messages.
- Important entry points: NetworkV62_BeginConnectEx, NetworkV60_BeginConnect, NetworkV60_Tick, NetworkV60_Disconnect, NetworkV60_SendChat, NetworkV60_SendAnimation, NetworkV60_SendHeldItemChange, NetworkV60_SendBlockDig, NetworkV60_SendPlace.
- Username and privacy: NetworkV62_SetUsername sanitizes the name; server address/IP is not used as username. Protocol mode uses NetworkV62_SetProtocolMode and NetworkV62_GetProtocolName.

Beta and Classic protocol paths

- Beta path sends handshake/login/position/look/chat/block dig/place and reads server packets through fixed-size parsing helpers: NetV60_ReadU8/I16/I32/Double/Float/String, NetV60_SkipItemStack, NetV60_SkipMetadata.
- Classic path uses NetV62Classic_SendIdentification, NetV62Classic_SendExtInfo, NetV62Classic_HandlePacket, NetV62Classic_ApplyLevel, and fixed 64-char Classic strings. It uses gzip/inflate style zlib functions for Classic level data.
- Packet terrain: NetV60_ApplyMapChunk decompresses Beta map chunk data. It converts server global coordinates into the active local window, writes world/g_blockMeta, then finishes with batch height/light/mesh rebuild.

zlib and server lag

- V74 includes DLLS/zlib.dll and the loader searches zlib1.dll, zlib.dll, DLLS\zlib1.dll, and DLLS\zlib.dll. If no compatible DLL is found, terrain packets are skipped safely instead of corrupting memory.
- Lag reduction comes from batch terrain apply: do not call SetBlock with full light/mesh rebuild per block. Server chunks should write arrays in bulk, then call RebuildColumnTops, ComputeLegacyLighting/local rebuild, and chunk invalidation once per batch.
- Network safety constants in the file cap buffers and ticks: NETV60_READ_BUFFER_SIZE, NETV60_MAX_DECOMPRESSED_CHUNK, NETV61_MAX_PACKETS_PER_TICK, NETV61_MAX_BYTES_PER_TICK, NETV60_READ_TIMEOUT.

Where to fix common multiplayer issues

- Crash on join: Ctrl+F NetV60_ParsePacket or packet ID case handling, NetV60_ApplyMapChunk, NetV62_LoadZlibInflate, NetV68_EnsureWindowForGlobal, NetV68_FinishRemoteChunkApply.
- Falling into void: check that server chunks are applied before gravity resumes; search g_netV60TerrainChunksApplied, NetV70_PrepareRemoteClientWorld, and player position correction packet handling.
- Bad chat/register: search NetworkV60_SendChat, NetV61_SanitizeOutgoingChat, GuiV9_HandleChar. Slash commands should be sent to the server while connected.

Open Watcom Compile Steps and Extension Checklist

Build requirements

- Use Open Watcom 2.x on Windows or a compatible Open Watcom environment. The project is a 32-bit Win32 OpenGL program and links against system OpenGL/WinMM/WinSock2 libraries.
- From a Watcom command prompt, enter the project root folder that contains project2finalalpharecreation.c and Makefile.v74.wat.

```
wmake -f Makefile.v74.wat
```

- Expected output: CloneMC_V74_JavaTerrain_Zlib_Performance.exe. The makefile runs: wcl386 -q -bt=nt -l=nt_win -fe=$(EXE) @clonemc_v74_nt_win.lnk project2finalalpharecreation.c
- Do not compile individual src/*.c files separately. They are included by the root file in a specific order. Do not add system nt_win inside the .lnk file if the makefile already passes -l=nt_win.

Environment setup example

```
set WATCOM=C:\WATCOM
set
PATH=%WATCOM%\binnt;%WATCOM%\binw;
%PATH% set
INCLUDE=%WATCOM%\h;%WATCOM%\h\nt
set LIB=%WATCOM%\lib386;%WATCOM%\lib386\nt
```

- If wmake is not found, PATH is wrong. If GL or WinSock symbols are missing, check clonemc_v74_nt_win.lnk and the Watcom LIB path.

Important Steps: If extracting the newest current test zip "CloneMCSingleMultiCodeTest.zip" is done and the code in src is edited and finished for compiling. Install Open Watcom C/C++ compiler, and in command prompt, run cd C:\Watcom, then run owsetenv.bat and then cd binnt64, and then from the extracted and code edited CloneMCSingleMultiCodeTest folder, take the project2finalalpha.c, clonemc_v74_nt_win link file, and Makefile.v74.wat and put it in binnt64 folder, binnt folder can also work if compiling in 32 bit systems. Then run "wmake -f Makefile.v74.wat" which should create a new executable and put it in the CloneMCtest + Makefile folder along with the asset folder. Make sure to delete the .obj and src folders related to the project in binnt64 if process is needed to start over, the makefiles and link files can be edited to change the output names of the executables created for example.

If the src folder in the github repo is edited and should be put in the second new test folder, CloneMCtest + Makefile, Install Open Watcom C/C++ compiler, and in command prompt, run cd C:\Watcom, then run owsetenv.bat and then cd binnt64, and then from the CloneMCtest + Makefile folder, take the src folder, project2finalalpha.c, clonemc_v58_nt_win link file, and Makefile.v58.wat and put it in binnt64 folder, binnt folder can also work if compiling in 32 bit systems. Then run "wmake -f Makefile.v58.wat" which should create an executable and put it in the CloneMCtest + Makefile folder along with the asset folder. Make sure to delete the .obj and src folders related to the project in binnt64 if a process is needed to start over, the makefiles and link files can be edited to change the output names of the executables created for example. project2finalalpharecreation.c used to be entire code in a c file, now it is just a linker file for the src folder.

Windows 98/XP notes

- The code uses classic OpenGL 1.1 immediate mode/display lists and Win32 APIs for old-system compatibility. WinSock2 is used, so ws2_32.lib and the WinSock2 runtime must exist.

- The included zlib.dll is a PE32/i386 DLL but may target newer Windows. For Windows 98, replace DLLS/zlib.dll with an older Win9x-compatible 32-bit zlib.dll or zlib1.dll.

Ctrl+F extension workflow

- Add a texture: search TILE_* and GetTerrainTileUV. Add a block: search BLOCK_* ID, BlockRegistryV19_Init, BlockV19_GetTextureTileAt, RenderBlockAtV19, GetBlockDropItemAt, BlockV50_CanPlaceBlockAt.
- Add a mob: search MOB_*, texMob*, LoadGameTextures, JavaModel_RenderExactMob, MobMaxHealthJavaV4, UpdateMobs, DropMobLoot, SaveMobToBytes.
- Add a particle: search PARTICLE_*, SpawnParticleV24, ParticleV54_GetParticleTile, ParticleV54_GetColorAndAlpha, RenderParticles.
- Add a GUI field/button: search g_state, LayoutBetaMenus, DrawButton2D, GuiV9TextField, WM_CHAR, WM_LBUTTONDOWN.
- Add a network packet: search packet ID cases, NetV60_Read*, NetV60_Write*, NetV60_SkipMetadata, NetworkV60_Tick. Always add length checks before reading bytes.

Safe coding rules for this project

- Keep C89 style for Open Watcom: declare variables at the start of blocks, avoid C++ references, avoid mixed declarations/statements, avoid
  // comments if an older compiler mode complains.
- When adding functions used earlier, add prototypes in src/00_core/clonemc_core_defs.c. When adding globals, define exactly once. When changing the include order, re-check root project2finalalpharecreation.c.
- After world/block edits, update height/light/meshes. After network chunk edits, batch changes instead of rebuilding per block. After asset edits, reload textures and confirm atlas coordinates.